ShaDe for SketchUp©

Loose subshape recognition: Technical Report

All rights reserved
For research use only (non-commercial use)
© Universidad de Málaga

Author	Manuela Ruiz Montiel, Fernando López Romero
Date	June 11, 2012
Version of ShaDe	4.0

0. Introduction

A **shape grammar** is a formalism that represents visual thinking. The proceeding way is the application of visual rules like the one in figure 1 to a given design. A grammar is a set of this kind of rules plus an initial design (a shape) called **axiom**.

Figure 1: A rule

These rules tell: "see the left side in a design and replace it with what is on the right". As we have said, the initial design is the axiom, and we start applying the rules from it, trying to match some sub-shape in the design with the left part of some rule. Then, we would replace the matched shape with what is on the right part of the rule, always applying to it the same transformation that we did in order to match the left shape. In figure 2 we can see an example of rule derivation, starting from a square as axiom:

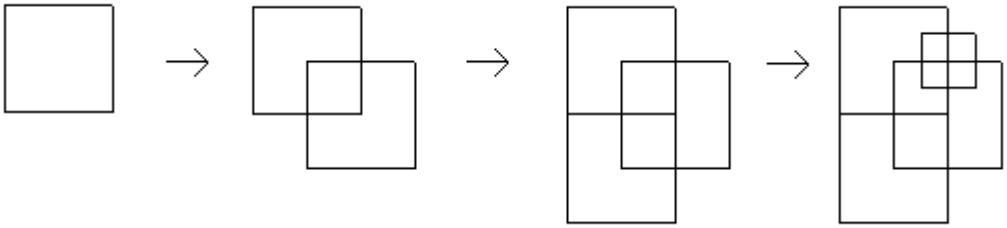


Figure 2: A rule derivation

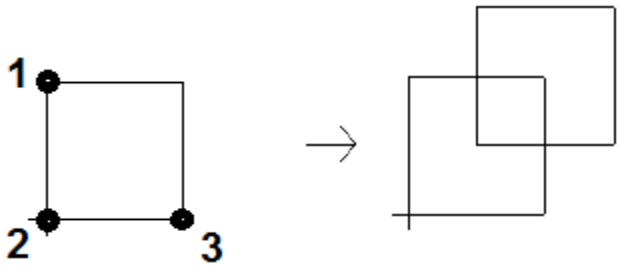
In order to apply the rules of the grammar, we can use the algorithm developed by Krishnamurti [1]. This method finds all the transformations that can be applied to the left part of a rule in order to *exactly* fit it inside the current design. The aim of this report is to illustrate how the original method can be modified in order to allow more flexible rule applications.

1. The original Krishnamurti's method

The method described in [1] is divided in a six-stage process. Here we briefly describe each step in order to keep the big picture. The working example is the one depicted in Figure 1, along with the axiom in Figure 2 (that is, the first shape in the derivation).

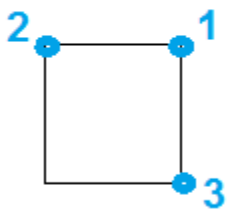
STEP 1: Select an ordered set of three points of the left part of the rule to apply.

For example:



STEP 2: Select an ordered set of three points of the current shape. If every set of three points has already been considered, then go to STEP 6.

For example:



STEP 3: The triangles formed by each set of points are compared in order to establish if they are equivalent. In order to do this, we compute the distances between the corresponding pairs of points and compare them:

$$D(\mathbf{1,2}) / D(1,2) = C1$$

$$D(\mathbf{1,3}) / D(1,3) = C2$$

$$D(\mathbf{2,3}) / D(2,3) = C3$$

If C1 = C2 = C3, then the triangles are equivalent and we move to STEP 4. If not, we go back to STEP 2.

STEP 4: Establish the affine transformation that is needed in order to transform the triangle of the left part of the rule into the triangle of the current shape.

STEP 5: Apply the computed transformation to the left part of the rule and test (with the algorithm detailed in [2]) if this is a subshape of the current design. If the test is passed, then add the transformation into the list of possible ones. If the test is not passed, go back to STEP 2.

STEP 6: Return the list of possible transformations that yield a subshape of the current design. **END OF THE ALGORITHM**.

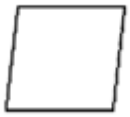
2. Loose subshape recognition algorithm

In the original shape grammar framework, sub-shapes inside a design are recognized exactly as they appear in the left part of the rules. Maybe they are rotated or scaled, but the shape itself must remain.

Our aim is to modify the algorithm in order to make it able to find transformations that yield sub-shapes that are slightly different from the pattern. For example, we may have the following axiom:

The original algorithm is not able to produce any transformation that converts the square of the left part of the rule into this trapezoidal axiom, because the needed transformation for that is not affine.

Moreover, if we consider an initial axiom like this one:



In which case the needed transformation is in fact affine, the original algorithm will still not produce any transformation, because the test inside STEP 3 would not be positive.

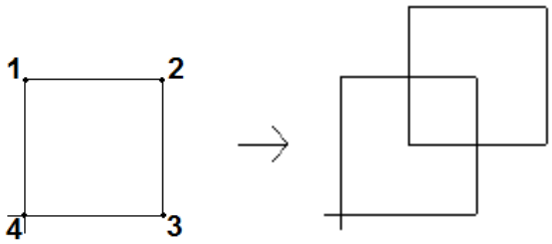
Nevertheless, we can argue that, albeit these axioms are not exactly squares, they are *similar* to a square. Therefore, we could still apply the rule and this will make sense.

In this section we describe the new algorithm that has been developed in order to recognize this type of transformations:

Let us suppose that there are K points inside the pattern.

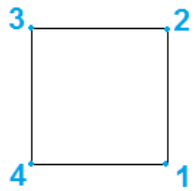
STEP 1: Select a permutation of the K points of the left part of the rule to apply.

For example:

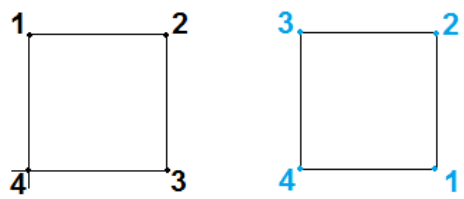


STEP 2: Select a *plausible* permutation of K points of the current shape. If every permutation of K points has already been considered, then go to STEP 5.

For example:



A permutation is defined as plausible if the angles formed by its points, taken in order, are *similar* to the angles that form the points of the pattern, also taken in order. Two angles a_1 and a_2 are similar if they differ less than ε, where ε is a parameterized value in radians; that is a_1 and a_2 are similar if $|a_1 - a_2| < \varepsilon$. In the running example, we have the following sets of points:



We must consider the following angles:

$\widehat{123}\ \widehat{234}\ \widehat{341}\ \widehat{412}$

$\widehat{123}\ \widehat{234}\ \widehat{341}\ \widehat{412}$

And compare them pairwise. In this case, all the angles are the same (90º) and thus the permutation is suitable.

STEP 3: Calculate the transformation that best transforms the set of points of the left part of the rule into the set of points of the current shape. The computed transformation will always be affine, that is, if we transform a point $\boldsymbol{p} = (p_1, p_2)^t$, the transformed point $\boldsymbol{p'} = (p'_1, p'_2)^t$ has the following form:

$$\boldsymbol{p}' = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \boldsymbol{p} + \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}$$

The coefficients of the matrices (a11, a12, a21, a22, b1 and b2) must be the ones that best transform all the points inside the pattern according to a least squares criterion, that is; we have to minimize the following expression *e*:

$$e = \sum_{k=1}^{K} d^2(\boldsymbol{p}_k{}', \boldsymbol{d}_k) = \sum_{k=1}^{K} ((p'_{kx} - d_{kx})^2 + (p'_{ky} - d_{ky})^2)$$

Where K is the total number of points inside the pattern, and $\boldsymbol{d}$ is a point inside the design permutation calculated in STEP 2. Notice that:

$$p'_{kx} = a_{11}p_{kx} + a_{12}p_{ky} + b_1$$

$$p'_{ky} = a_{21}p_{kx} + a_{22}p_{ky} + b_2$$

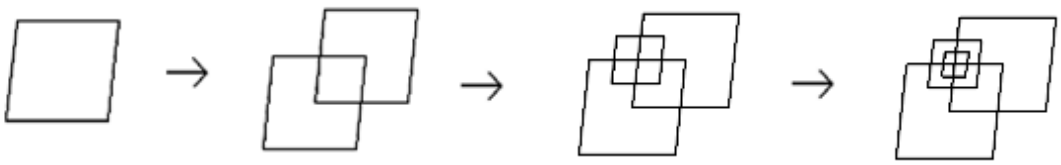
STEP 4: Apply the computed transformation to the left part of the rule and test (with the method described in section 3) if this is a subshape of the current design. If the test is passed, then add the transformation into the list of possible ones. If the test is not passed, go back to STEP 2.

STEP 5: Return the list of possible transformations that yield a subshape of the current design. **END OF THE ALGORITHM**.

We illustrate two possible derivations starting from axioms that are not exactly squares.

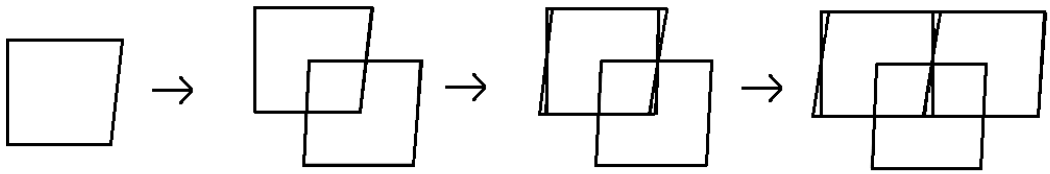
For example, if we have the following axiom:

With ε = 0.2 we may produce the following derivation:



If we have the following axiom:

We may produce the following derivation, also with ε = 0.2:



We can see how, in all the applications, the algorithm does not restrict to exact instances of a square inside the design.

3. Subshape testing

Once the transformation has been computed, we need to assure that the yielded shape is in fact a subshape of the current design. To do this, we have implemented the following test for each pair of points inside the pattern:

"If two points of the pattern permutation are connected inside the pattern, then their corresponding points of the design permutation calculated in STEP2 are also connected inside the design".

We understand that two points p_1 and p_2 are connected inside a shape, if there exist a segment such that their end points are p_1 and p_2.

4. Shape degrees

When a transformation is calculated in STEP 3, an error e is associated with it. If we define the diameter d of a shape as the maximum distance between two points inside the shape, then we can define a degree of inaccuracy:

$$\mu = \frac{e}{d}$$

So the shape that is produced by the application of such transformation has a new degree, defined by the following expression:

$$\mu_{new} = \max(\mu, \mu_{old})$$

5. Improving performance

The original algorithm computes all the possible transformations that yield a subshape of the current design. Nevertheless, we can stop the algorithm in STEP 4, once the subshape test is passed for the first time.